

- test/review_practice 代码总结
 - 1. 目录结构总览
 - graph/
 - tree/
 - leetcode/
 - 2. 图论部分总结
 - graph/buildtheinternet.cpp
 - graph/buildtheinternet2.cpp
 - graph/bfs.cpp
 - graph/bfs2.cpp
 - graph/color.cpp
 - graph/findAllisland.cpp
 - graph/maxpath.cpp
 - graph/shortestpathlen.cpp
 - 3. 树部分总结
 - tree/re_serialize.cpp
 - tree/max_width_tree.cpp
 - tree/longestdepth.cpp
 - tree/longestwayintree.cpp
 - tree/maxpathsum.cpp
 - tree/buildfulltreewithfrontandback.cpp
 - 4. LeetCode 练习部分总结
 - leetcode/寻找图中是否存在路径.cpp
 - leetcode/不同颜色的路径.cpp
 - leetcode/访问所有节点的最短路径.cpp
 - leetcode/相邻字符的不同路径长度.cpp
 - leetcode/二叉树的直径.cpp
 - leetcode/计算递增路径数量.cpp
 - 5. 和期末提示的对应关系
 - 图结构解决实际问题
 - 树的重构与树上算法
 - 6. 建议的复习顺序
 - 7. 核心解题思路总结
 - 图论题常见判断方法
 - 1. 看到“最短步数、最少操作次数、迷宫最短路”
 - 2. 题目要求“输出最短路具体经过哪些点”
 - 3. 看到“连通块、岛屿数量、是否连通”

- 4. 看到“所有点都要连起来，而且总代价尽量小”
 - 5. 看到“状态不只和点有关，还和访问集合 / 颜色 / 上一步有关”
 - 6. 看到“尝试所有方案、检查是否合法、求最小可行值”
- 树题常见判断方法
 - 1. 看到“根据遍历序列恢复二叉树”
 - 2. 看到“最大深度、树高”
 - 3. 看到“最长路径、直径”
 - 4. 看到“最大路径和”
 - 5. 看到“树的宽度、按层处理”
- 8. 经常复用的代码模板
 - 1. 图的邻接表模板
 - 2. 网格 BFS 模板
 - 3. BFS 路径还原模板
 - 4. 连通块 DFS 模板
 - 5. Prim 最小生成树模板
 - 6. 树的基础递归模板
 - 7. 树的直径模板
 - 8. 树的最大路径和模板
 - 9. 二叉树层序遍历模板
 - 10. 先序反序列化模板
- 9. 考前最值得背下来的几个结论
- 10. 一句话总结
- 11. 逐题详细解题过程
 - graph/buildtheinternet.cpp
 - 题目本质
 - 为什么这样建模
 - 为什么用 Prim
 - 代码过程
 - 容易错的地方
 - graph/buildtheinternet2.cpp
 - 题目本质
 - 复习时应该抓什么
 - 这题的真正价值
 - graph/bfs.cpp
 - 题目本质
 - 为什么建成网格图
 - 为什么用 BFS
 - 代码过程

- 容易错的地方
- graph/bfs2.cpp
 - 题目本质
 - 和 bfs.cpp 的区别
 - 代码过程
 - 为什么这样恢复一定是一条最短路
 - 容易错的地方
- graph/color.cpp
 - 题目本质
 - 为什么用 DFS + 回溯
 - 代码过程
 - 这题在练什么
 - 容易错的地方
- graph/findAllisland.cpp
 - 题目本质
 - 为什么用 DFS
 - 代码过程
 - 为什么能这样原地修改
 - 容易错的地方
- graph/maxpath.cpp
 - 题目本质
 - 为什么可以用 DFS + 记忆化
 - 代码过程
 - 这题在练什么
 - 容易错的地方
- graph/shortestpathlen.cpp
 - 题目本质
 - 为什么要引入 mask
 - 为什么还是 BFS
 - 代码过程
 - 这题在练什么
 - 容易错的地方
- tree/re_serialize.cpp
 - 题目本质
 - 为什么先序遍历 + null 可以唯一恢复
 - 序列化过程
 - 反序列化过程
 - 为什么 idx 必须按引用传递

- 容易错的地方
- tree/max_width_tree.cpp
 - 题目本质
 - 为什么普通层序遍历不够
 - 为什么用位置编号
 - 代码过程
 - 容易错的地方
- tree/longestdepth.cpp
 - 题目本质
 - 为什么递归这么自然
 - 代码过程
 - 这题的意义
- tree/longestwayintree.cpp
 - 题目本质
 - 最关键的思想
 - 代码过程
 - 为什么返回值不能直接写成 $L + R + 1$
 - 容易错的地方
- tree/maxpathsum.cpp
 - 题目本质
 - 为什么和直径题像
 - 代码过程
 - 为什么负数贡献要舍弃
 - 最核心的理解
- tree/buildfulltreewithfrontandback.cpp
 - 题目本质
 - 为什么这题比“前序 + 中序”难
 - 代码的思路
 - 这题为什么值得复习
- leetcode/寻找图中是否存在路径.cpp
 - 题目本质
 - 为什么用 BFS
 - 代码过程
 - 为什么最后返回 `visited[destination]` 更稳
- leetcode/不同颜色的路径.cpp
 - 题目本质
 - 为什么普通最短路不够
 - 所以状态怎么设计

- 代码过程
- 这题的本质训练点
- leetcode/访问所有节点的最短路径.cpp
 - 题目本质
 - 为什么不能用普通单源最短路
 - 代码过程理解
 - 复习时重点背什么
- leetcode/相邻字符的不同路径长度.cpp
 - 题目本质
 - 为什么还是树形 DFS
 - 代码过程
 - 核心思想
- leetcode/二叉树的直径.cpp
 - 题目本质
 - 应该怎么理解这题
 - 这题和树高题的关系
- leetcode/计算递增路径数量.cpp
 - 题目本质
 - 为什么可以按数值排序做 DP
 - 代码过程
 - 这题在练什么

test/review_practice 代码总结

这份目录里的代码可以大致分成三类：

1. 图论基础与图上的实际问题
2. 二叉树重构与树上递归 / 层序算法
3. LeetCode 风格的图树扩展题

整体上和期末提示非常贴合：

- 图部分重点在“如何从输入构图、如何存图、如何遍历、如何求最短路/连通性/路径代价”
 - 树部分重点在“如何根据序列重构树、如何遍历、如何计算深度/宽度/路径长度”
-

1. 目录结构总览

graph/

- `buildtheinternet.cpp`
- `buildtheinternet2.cpp`
- `bfs.cpp`
- `bfs2.cpp`
- `color.cpp`
- `findAllisland.cpp`
- `maxpath.cpp`
- `shortestpathlen.cpp`

tree/

- `buildfulltreewithfrontandback.cpp`
- `longestdepth.cpp`
- `longestwayintree.cpp`
- `maxpathsum.cpp`
- `max_width_tree.cpp`
- `re_serialize.cpp`

leetcode/

- `不同颜色的路径.cpp`
- `二叉树的直径.cpp`
- `寻找图中是否存在路径.cpp`
- `相邻字符的不同路径长度.cpp`
- `计算递增路径数量.cpp`
- `访问所有节点的最短路径.cpp`

2. 图论部分总结

这一部分最值得重点复习，因为它最贴近期末提示里的“用图结构解决实际问题”。

graph/buildtheinternet.cpp

主题：最小生成树 + 树上 DFS

代码内容：

- 输入若干岛屿的坐标和人口
- 用点之间的欧几里得距离作为边权
- 通过 Prim 算法建立最小生成树
- 再把最小生成树当作一棵树，DFS 计算每个岛的接入时间
- 最终按人口加权，计算平均接入时间

核心知识点：

- 从实际问题抽象成带权完全图
- 不显式存全部边，而是在 Prim 过程中动态计算距离
- `parent[]` 记录最小生成树结构
- 树上 DFS 不一定求“路径和”，这里求的是“根到当前点路径上的最大边权”

这题很值得看，因为它把：

- 建模
- 图算法
- 树遍历
- 结果统计

串到了一起，是非常典型的综合题。

graph/buildtheinternet2.cpp

主题：`buildtheinternet.cpp` 的同类实现

代码内容基本相同，也是：

- Prim 建最小生成树
- DFS 统计树上从根到各点的接入时间

可以把它当成同一题的第二份练习稿，复习时重点还是掌握：

- `minDist[]`
- `parent[]`
- `used[]`
- 最小生成树建出来以后如何转成邻接表再 DFS

graph/bfs.cpp

主题：迷宫 / 网格最短路 BFS

代码内容：

- 在一个字符网格中找从起点到终点的最短步数
- 网格中的 `*` 表示不可走
- `|` 和 `-` 对移动方向有限制
- BFS 求最短路长度

核心知识点：

- 网格图建模
- 队列实现 BFS
- `dist[][]` 记录最短步数
- BFS 第一次到达某点时就是最短路
- 不仅要判断“能不能走到目标格”，还要判断“当前格是否允许朝该方向走出”

这题和期末提示里的“迷宫问题”非常接近，建议重点复习。

graph/bfs2.cpp

主题：最短路 + 路径还原

在 `bfs.cpp` 的基础上增加了：

- `pre[][]` 前驱数组
- 到达终点后反向回溯整条路径
- 输出完整经过的坐标序列

核心知识点：

- 最短路长度和最短路径本身是两个层次
- 如果题目要求输出路径，BFS 时必须额外记录前驱

- 路径恢复一般是“终点不断跳前驱直到起点，再反转”

如果考试不只问“最短距离”，而是问“具体怎么走”，这份代码就很有参考价值。

graph/color.cpp

主题：图着色 / DFS 回溯

代码内容：

- 输入图的邻接关系
- 尝试用 1 到 4 种颜色给图染色
- 相邻节点颜色不能相同
- 用 DFS + 回溯判断最少需要几种颜色

核心知识点：

- 邻接矩阵存图
- 回溯枚举每个点的颜色
- 冲突检测
- 状态撤销

这是“图遍历 + 搜索”的代表题，虽然不一定是期末主方向，但很适合练 DFS 和状态设计。

graph/findAllisland.cpp

主题：连通块 / 岛屿数量

代码内容：

- 在二维 0/1 网格中
- 遇到一个陆地格子就 DFS 扩散
- 把整个连通块标记掉
- 统计岛屿数量

核心知识点：

- 二维网格 DFS
- flood fill

- 连通分量统计

这是图遍历最基础的模型之一，适合和迷宫 BFS 一起记。

graph/maxpath.cpp

主题：矩阵中的最长递增路径 + 记忆化 DFS

代码内容：

- 在矩阵中从某格出发，只能走向更大的相邻格
- DFS 搜索最长递增路径
- `dp[x][y]` 记忆化，避免重复计算
- 还额外保存了具体路径

核心知识点：

- 有向无环搜索思想
- 记忆化 DFS
- “最长路径”不一定只能用普通 DP，也可以用 DFS + 记忆化

graph/shortestpathlen.cpp

主题：访问所有节点的最短路径长度

代码内容：

- 每个状态是 (当前点, 已访问节点集合mask, 当前长度)
- 从每个点都可以作为起点
- 用 BFS 在“状态图”上找最短路

核心知识点：

- 状态压缩
- `mask` 表示访问集合
- BFS 不只跑在原图上，也可以跑在“扩展状态图”上

这是比较进阶的图论题，适合在基础 BFS 熟练以后看。

3. 树部分总结

这一部分和期末提示里的“树的重构以及树结构上的相关算法”对应得非常直接。

tree/re_serialize.cpp

主题：二叉树的序列化与反序列化

代码内容：

- 用先序遍历把树转成字符串
- 空节点记成 `null`
- 反序列化时按照同样的先序顺序递归恢复

核心知识点：

- 先序遍历序列 + 空指针标记，能够唯一恢复一棵二叉树
- 反序列化时要共享一个“当前读到第几个 token”的下标
- 构造顺序是：根 -> 左子树 -> 右子树

这份代码非常值得重点复习，因为它就是典型的“根据 DFS 序列重构树”。

tree/max_width_tree.cpp

主题：二叉树最大宽度

代码内容：

- 层序遍历二叉树
- 队列里除了节点，还保存“完全二叉树下标”
- 每层宽度 = 这一层最右下标 - 最左下标 + 1

核心知识点：

- BFS 层序遍历
- 队列分层处理
- 用位置编号解决“中间空缺节点也算宽度”的问题

这题和你给出的提示“建议仔细研究教材中关于求二叉树宽度的示例代码”几乎完全对应，应该列为重点中的重点。

tree/longestdepth.cpp

主题：二叉树最大深度

代码内容：

- 递归求左子树深度
- 递归求右子树深度
- 返回两者较大值加一

核心知识点：

- 树的后序递归思想
- “先求子问题，再合并答案”

这是树递归最基础的模板。

tree/longestwayintree.cpp

主题：二叉树直径 / 最长路径

代码内容：

- 递归求左右子树深度
- 用 $L + R + 1$ 更新全局最长路径
- 返回“从当前节点向下延伸的最大深度”

核心知识点：

- 返回值和全局答案可以不是同一个含义
- 递归返回的是“向下的最长链”
- 真正答案可能经过当前根节点，等于“左高 + 右高 + 当前点”

这类题是树上递归的经典模型。

tree/maxpathsum.cpp

主题：二叉树最大路径和

代码内容：

- 每个节点考虑左右子树能提供的最大正贡献
- 如果子路径和为负，就舍弃
- 更新“经过当前节点”的最大路径和

核心知识点：

- 树上动态规划思想
- 路径问题里常见的“只保留正贡献”
- 与“二叉树直径”很像，但比较的是权值和，不是边数/节点数

tree/buildfulltreewithfrontandback.cpp

主题：由前序 + 后序重构二叉树

代码内容：

- 想法是通过前序确定根
- 再用前序第二个元素确定左子树根
- 去后序中找这个左子树根的位置，从而分出左右子树范围

需要注意：

- 这份代码当前还是未完成状态，`findIndex` 还没有真正实现
- 它针对的是“满二叉树/可唯一划分的情况”，不是任意二叉树都能唯一重构

虽然这份代码不完整，但它的思路很适合复习“树重构”。

4. LeetCode 练习部分总结

这一部分相当于对图和树主干知识的补充强化。

leetcode/寻找图中是否存在路径.cpp

主题：无向图连通性判断

代码内容：

- 邻接表建图
- BFS 从 **source** 出发
- 判断能否到达 **destination**

核心知识点：

- 邻接表
- BFS/visited
- 连通性查询

leetcode/不同颜色的路径.cpp

主题：交替颜色最短路

代码内容：

- 红边图和蓝边图分开存
- BFS 状态里不仅有节点，还有“上一条边颜色”
- 从而保证下一步必须走另一种颜色的边

核心知识点：

- 扩展状态 BFS
- **dist[node][color]**
- “最短路条件里多了一维状态”

leetcode/访问所有节点的最短路径.cpp

主题：状态压缩 BFS

代码内容和 **graph/shortestpathlen.cpp** 本质一致：

- 状态是 (节点, **mask**, 距离)
- 从所有节点同时作为起点
- 找到访问全集时的最短长度

这是图最短路中的高频进阶模型。

leetcode/相邻字符的不同路径长度.cpp

主题：树上的最长合法路径

代码内容：

- 由 `parent[]` 建树/无向图
- DFS 每个节点的子树
- 只把“相邻字符不同”的子路径接起来
- 维护最长链和全局答案

核心知识点：

- 树形 DFS
- 最长链合并
- 约束条件下的树上路径问题

leetcode/二叉树的直径.cpp

主题：二叉树直径

和 `tree/longestwayintree.cpp` 是同一类问题：

- DFS 求左右深度
- 用左右深度和更新答案
- 返回单边最大深度

适合和树目录下那份代码对照看。

leetcode/计算递增路径数量.cpp

主题：统计严格递增路径条数

代码内容：

- 把所有格子按数值从小到大排序
- `dp[x][y]` 表示以当前格结尾的递增路径数
- 枚举比当前值小的邻居进行转移

核心知识点：

- 图上的 DP 思想
 - 利用值大小规律做“拓扑序式”转移
 - 计数型动态规划
-

5. 和期末提示的对应关系

根据你给出的提示，这个目录里最值得优先复习的代码如下。

图结构解决实际问题

优先级最高：

- `graph/bfs.cpp`
- `graph/bfs2.cpp`
- `graph/buildtheinternet.cpp`
- `leetcode/寻找图中是否存在路径.cpp`

重点掌握：

- 如何把输入转成邻接表 / 网格图
- BFS 的队列写法
- `visited` / `dist` 的含义
- 最短路为什么能用 BFS
- 如果要求输出路径，如何增加前驱数组
- 带权问题和无权问题的区别

可作为扩展提高：

- `graph/findAllisland.cpp`
- `graph/color.cpp`
- `graph/shortestpathlen.cpp`
- `leetcode/不同颜色的路径.cpp`
- `leetcode/访问所有节点的最短路径.cpp`

树的重构与树上算法

优先级最高：

- `tree/re_serialize.cpp`
- `tree/max_width_tree.cpp`
- `tree/longestdepth.cpp`
- `tree/longestwayintree.cpp`

重点掌握：

- 先序遍历如何重构树
- 递归函数的返回值代表什么
- 全局答案和递归返回值的区别
- 层序遍历如何一层一层处理
- 宽度为什么是“最右位置 - 最左位置 + 1”

可作为扩展提高：

- `tree/maxpathsum.cpp`
- `tree/buildfulltreewithfrontandback.cpp`
- `leetcode/二叉树的直径.cpp`
- `leetcode/相邻字符的不同路径长度.cpp`

6. 建议的复习顺序

如果按期末复习效率来排，建议顺序如下：

1. `graph/bfs.cpp`
2. `graph/bfs2.cpp`
3. `leetcode/寻找图中是否存在路径.cpp`
4. `graph/buildtheinternet.cpp`
5. `tree/longestdepth.cpp`
6. `tree/longestwayintree.cpp`
7. `tree/re_serialize.cpp`
8. `tree/max_width_tree.cpp`

然后再看扩展题：

1. `graph/findAllisland.cpp`
2. `graph/color.cpp`
3. `graph/shortestpathlen.cpp`
4. `tree/maxpathsum.cpp`

- 5. [leetcode/不同颜色的路径.cpp](#)
 - 6. [leetcode/访问所有节点的最短路径.cpp](#)
 - 7. [leetcode/计算递增路径数量.cpp](#)
-

7. 核心解题思路总结

这一部分更适合考前背诵，重点不是某一道题，而是“看到题以后怎么判断用什么方法”。

图论题常见判断方法

1. 看到“最短步数、最少操作次数、迷宫最短路”

优先想到：BFS

判断依据：

- 每次移动代价相同
- 问“最少几步”
- 图可能是普通图，也可能是二维网格

对应代码：

- [graph/bfs.cpp](#)
- [graph/bfs2.cpp](#)
- [leetcode/寻找图中是否存在路径.cpp](#)

思路口诀：

- 先建图
- 起点入队
- 用 `dist` 或 `visited` 防止重复访问
- 第一次到某点就是最短路

2. 题目要求“输出最短路具体经过哪些点”

优先想到：BFS + 前驱数组 `pre`

判断依据：

- 不只是问距离，还要输出路径
- 需要知道每个点是从哪里转移来的

对应代码：

- `graph/bfs2.cpp`

思路口诀：

- 搜索时记录 `pre[next] = cur`
- 从终点沿 `pre` 倒推回起点
- 最后反转路径

3. 看到“连通块、岛屿数量、是否连通”

优先想到：`DFS / BFS` 遍历连通分量

判断依据：

- 题目本质是在问“有几块”
- 或者问“两个点能不能互相到达”

对应代码：

- `graph/findAllisland.cpp`
- `leetcode/寻找图中是否存在路径.cpp`

思路口诀：

- 遍历所有点
- 遇到未访问点就启动一次 DFS/BFS
- 一次搜索吃掉一个连通块

4. 看到“所有点都要连起来，而且总代价尽量小”

优先想到：`最小生成树 MST`

判断依据：

- 关键词通常是“连通全部点”
- “总成本最小”

- “修路 / 铺线 / 联网 / 供电”

对应代码：

- `graph/buildtheinternet.cpp`
- `graph/buildtheinternet2.cpp`

思路口诀：

- 如果是稠密图，Prim 很常见
- `minDist[]` 表示当前点到生成树的最短边
- `parent[]` 记录这条最优边从哪来

5. 看到“状态不只和点有关，还和访问集合 / 颜色 / 上一步有关”

优先想到：扩展状态 BFS

判断依据：

- 题目里的状态多了一维
- 比如“访问过哪些点”“上一步走的是什么颜色”“当前持有什么状态”

对应代码：

- `graph/shortestpathlen.cpp`
- `leetcode/访问所有节点的最短路径.cpp`
- `leetcode/不同颜色的路径.cpp`

思路口诀：

- 原图的一个点不再是完整状态
- 状态要写成 (点, 附加信息)
- `visited / dist` 也要按完整状态记录

6. 看到“尝试所有方案、检查是否合法、求最小可行值”

优先想到：DFS + 回溯

判断依据：

- 题目让你枚举颜色、分组、选法
- 每一步都要检查约束是否合法

对应代码：

- `graph/color.cpp`

思路口诀：

- 先尝试
 - 不合法就剪枝
 - 回来时撤销现场
-

树题常见判断方法

1. 看到“根据遍历序列恢复二叉树”

优先想到：递归建树

判断依据：

- 给出先序 / 中序 / 后序
- 或者给出序列化字符串
- 问能否重建原树

对应代码：

- `tree/re_serialize.cpp`
- `tree/buildfulltreewithfrontandback.cpp`

思路口诀：

- 先确定根
- 再划分左右子树范围
- 递归构造左右子树

2. 看到“最大深度、树高”

优先想到：树的递归深度函数

对应代码：

- `tree/longestdepth.cpp`

思路口诀：

- 空树返回 0
- 左右子树分别求深度
- 返回 $\max(\text{left}, \text{right}) + 1$

3. 看到“最长路径、直径”

优先想到：递归返回单边最长链 + 全局维护答案

对应代码：

- `tree/longestwayintree.cpp`
- `leetcode/二叉树的直径.cpp`
- `leetcode/相邻字符的不同路径长度.cpp`

思路口诀：

- 递归返回的是“从当前点往下走的一条最长链”
- 真正答案可能经过当前点，把左右两边接起来
- 所以要有一个全局变量维护答案

4. 看到“最大路径和”

优先想到：树上 DP / 递归贡献值

对应代码：

- `tree/maxpathsum.cpp`

思路口诀：

- 子树如果贡献是负数，就不要
- 当前点能向父节点提供的，只能是一条单边路径
- 但更新全局答案时，可以左右都取

5. 看到“树的宽度、按层处理”

优先想到：层序遍历 BFS

对应代码：

- `tree/max_width_tree.cpp`

思路口诀：

- 队列分层
- 每层单独统计
- 如果题目里的宽度包含中间空位，就加位置编号

8. 经常复用的代码模板

下面这些模板和当前目录里的代码非常对应，考试时很容易直接套。

1. 图的邻接表模板

```
vector<vector<int>> adj(n);
for (auto &e : edges)
{
    int u = e[0], v = e[1];
    adj[u].push_back(v);
    adj[v].push_back(u); // 无向图要加这句
}
```

适用场景：

- 普通无向图
- 连通性判断
- BFS / DFS 遍历

2. 网格 BFS 模板

```
queue<pair<int, int>> q;
vector<vector<int>> dist(n, vector<int>(m, -1));
int dx[4] = {-1, 1, 0, 0};
int dy[4] = {0, 0, -1, 1};

q.push({sx, sy});
dist[sx][sy] = 0;

while (!q.empty())
{
    auto [x, y] = q.front();
    q.pop();
```

```

for (int k = 0; k < 4; k++)
{
    int nx = x + dx[k];
    int ny = y + dy[k];

    if (nx < 0 || nx >= n || ny < 0 || ny >= m)
        continue;
    if (dist[nx][ny] != -1)
        continue;
    if (!canMove(nx, ny))
        continue;

    dist[nx][ny] = dist[x][y] + 1;
    q.push({nx, ny});
}
}

```

对应代码：

- [graph/bfs.cpp](#)

3. BFS 路径还原模板

```

vector<vector<pair<int, int>>> pre(n, vector<pair<int, int>>(m, {-1, -1}));

// 转移时
pre[nx][ny] = {x, y};

// 回溯路径
vector<pair<int, int>> path;
pair<int, int> cur = {tx, ty};
while (cur.first != -1)
{
    path.push_back(cur);
    cur = pre[cur.first][cur.second];
}
reverse(path.begin(), path.end());

```

对应代码：

- [graph/bfs2.cpp](#)

4. 连通块 DFS 模板

```

void dfs(int u)
{
    visited[u] = true;
    for (int v : adj[u])
    {
        if (!visited[v])
            dfs(v);
    }
}

int cnt = 0;
for (int i = 0; i < n; i++)
{
    if (!visited[i])
    {
        cnt++;
        dfs(i);
    }
}

```

网格版本只需要把 u 换成 (x, y) 。

对应代码：

- [graph/findAllisland.cpp](#)
- [leetcode/寻找图中是否存在路径.cpp](#)

5. Prim 最小生成树模板

```

vector<double> minDist(n, INF);
vector<int> parent(n, -1);
vector<bool> used(n, false);

minDist[0] = 0;

for (int i = 0; i < n; i++)
{
    int u = -1;
    double best = INF;
    for (int j = 0; j < n; j++)
    {
        if (!used[j] && minDist[j] < best)
        {
            best = minDist[j];
            u = j;
        }
    }

    used[u] = true;
}

```

```

for (int v = 0; v < n; v++)
{
    if (used[v] || v == u)
        continue;
    double w = getDist(u, v);
    if (w < minDist[v])
    {
        minDist[v] = w;
        parent[v] = u;
    }
}
}

```

对应代码：

- [graph/buildtheinternet.cpp](#)

6. 树的基础递归模板

```

int dfs(TreeNode *root)
{
    if (root == nullptr)
        return 0;

    int L = dfs(root->left);
    int R = dfs(root->right);

    return max(L, R) + 1;
}

```

适用场景：

- 最大深度
- 高度类问题

对应代码：

- [tree/longestdepth.cpp](#)

7. 树的直径模板

```

int ans = 0;

```

```
int dfs(TreeNode *root)
{
    if (root == nullptr)
        return 0;

    int L = dfs(root->left);
    int R = dfs(root->right);

    ans = max(ans, L + R);
    return max(L, R) + 1;
}
```

关键点：

- 返回值是“单边最长链”
- `ans` 才是最终直径

对应代码：

- `tree/longestwayintree.cpp`
- `leetcode/二叉树的直径.cpp`

8. 树的最大路径和模板

```
int ans = INT_MIN;

int dfs(TreeNode *root)
{
    if (root == nullptr)
        return 0;

    int L = max(dfs(root->left), 0);
    int R = max(dfs(root->right), 0);

    ans = max(ans, L + R + root->val);
    return max(L, R) + root->val;
}
```

对应代码：

- `tree/maxpathsum.cpp`

9. 二叉树层序遍历模板

```

queue<TreeNode*> q;
q.push(root);

while (!q.empty())
{
    int sz = q.size();
    for (int i = 0; i < sz; i++)
    {
        TreeNode *cur = q.front();
        q.pop();

        if (cur->left) q.push(cur->left);
        if (cur->right) q.push(cur->right);
    }
}

```

如果题目要求最大宽度，就在队列里连同位置编号一起存。

对应代码：

- [tree/max_width_tree.cpp](#)

10. 先序反序列化模板

```

TreeNode* build(vector<string> &nodes, int &idx)
{
    if (idx >= nodes.size() || nodes[idx] == "null")
    {
        idx++;
        return nullptr;
    }

    TreeNode *root = new TreeNode(stoi(nodes[idx++]));
    root->left = build(nodes, idx);
    root->right = build(nodes, idx);
    return root;
}

```

关键点：

- 一定要把 `idx` 按引用传下去
- 顺序必须和序列化时保持一致

对应代码：

- [tree/re_serialize.cpp](#)

9. 考前最值得背下来的几个结论

1. 无权最短路优先想 **BFS**，不是 **DFS**。
 2. 要输出路径，搜索时就要记录前驱，不能最后再想。
 3. 连通块问题本质就是“从一个未访问点出发，吃掉整块区域”。
 4. 最小生成树适合“把所有点连起来且总代价最小”。
 5. 树的递归题要先分清：返回值表示什么，全局变量表示什么。
 6. 树的直径 / 最长路径类题，返回值通常是“单边最长链”。
 7. 树的宽度如果包含空位，就要给节点编号，不能只数节点个数。
 8. 根据先序序列重构树时，根一定先被读到，然后递归构造左子树和右子树。
-

10. 一句话总结

test/review_practice 这个目录本质上就是一套围绕“图的遍历与最短路”和“树的重构与路径/宽度计算”展开的复习题集，其中：

- 图部分最像考试的是迷宫 BFS、连通性判断、最小生成树建模
- 树部分最像考试的是先序重构、树高/直径、二叉树宽度

如果期末题目真的只偏向“图实际问题 + 树重构/宽度”，那么这份目录里的重点文件已经很集中。

11. 逐题详细解题过程

这一节按“题目本质 -> 为什么这样建模 -> 为什么选这个算法 -> 代码是怎么一步一步做的 -> 容易错在哪”来展开。

graph/buildtheinternet.cpp

题目本质

这道题表面上是在“岛屿之间铺设网络”，本质上是两层问题：

1. 怎样把所有岛连起来，并且铺设总代价尽量小
2. 连好以后，每个岛真正接入网络的时间怎么算

第一层是典型最小生成树问题，第二层是树上的路径统计问题。

为什么这样建模

每个岛可以看成一个点，任意两个岛之间都可以连线，所以这实际上是一个完全图。

- 点：岛屿
- 边：两个岛之间的连线方案
- 边权：两岛的欧几里得距离

因为“要把所有点连起来，而且总代价尽量小”，所以自然想到最小生成树。

为什么用 Prim

这题没有提前把所有边存下来，而是直接通过坐标现算距离。由于任意两点之间都可能连边，图比较稠密，用朴素 Prim 很顺手：

- `minDist[i]`：点 `i` 到当前生成树的最短距离
- `parent[i]`：这个最短距离是从哪个点连过来的
- `used[i]`：点 `i` 是否已经加入生成树

代码过程

第一步，读入所有岛的坐标和人口。

第二步，初始化 Prim：

- 把 0 号岛当成主岛
- `minDist[0] = 0`
- 其他点到生成树的距离先设成无穷大

第三步，做 `n` 轮 Prim：

- 每次从还没加入生成树的点中选一个 `minDist` 最小的点 `u`
- 把 `u` 加入生成树
- 用 `u` 去更新其他点到生成树的最短距离

第四步，根据 `parent[]` 把最小生成树真正建出来，转成邻接表 `tree`。

第五步，在这棵树上 DFS。

这里最关键的一点是：

- 不是把根到当前点路径上的边权相加
- 而是取这条路径上的最大边权

原因是题意里“所有光缆同时铺设”，所以一个岛的接入时间不是路径总长度，而是这条路径上最慢完成的那段线缆长度。

所以 DFS 传递的是：

- `curTime`：从根到当前点路径上的最大边权

到子节点时更新为：

- `max(curTime, w)`

最后把每个岛的接入时间乘以人口，累加后除以总人口，得到加权平均时间。

容易错的地方

1. 很容易误写成路径和 `curTime + w`，但这题应该是 `max(curTime, w)`。
2. 最小生成树建完以后，别忘了它已经是一棵树，后续要用树的思路，不再是原始完全图。
3. `parent[]` 只记录生成树结构，真正 DFS 前还要把它转成双向邻接表。

graph/buildtheinternet2.cpp

题目本质

和上一题本质相同，也是：

- 先求最小生成树
- 再在树上统计接入时间

复习时应该抓什么

这一份更像是同一题的再练习，所以重点不在结论，而在于把流程记熟：

1. 点的几何距离怎么算

2. Prim 的三个核心数组怎么定义
3. `parent[]` 如何还原成树
4. 树上 DFS 如何传递“到当前点的状态”

这题的真正价值

不是背代码细节，而是形成一个综合题框架：

- 几何建图
- MST
- 邻接表
- DFS 统计

以后看到“先最优连通，再在树上算别的量”的题，都可以联想到这题。

graph/bfs.cpp

题目本质

这是一个网格最短路问题。问从起点走到终点最少多少步。

因为每走一步代价都一样，所以这是标准 BFS 场景。

为什么建成网格图

矩阵中的每个格子都可以看成一个点。

- 上下左右四个方向的移动就是边
- * 表示障碍，不能经过
- | 和 - 不是普通空地，它们对移动方向有限制

所以这题不是单纯“能不能走”，而是“当前格能不能朝这个方向走出去，目标格能不能从这个方向走进来”。

为什么用 BFS

因为：

- 边权都相同，都是一步
- 要求最少步数

BFS 按层扩展，先到达的一定步数更小，所以第一次到终点时就是最短路。

代码过程

第一步，读入网格大小、起点终点、地图。

第二步，把题目中的 1-based 坐标改成 0-based，方便和数组下标统一。

第三步，定义：

- `dist[x][y]`：起点到 (x, y) 的最短步数
- 初始全为 `-1`，表示没访问过

第四步，起点入队，`dist[start] = 0`。

第五步，开始 BFS：

- 队头取一个格子
- 枚举四个方向
- 判断新坐标是否越界
- 判断是否访问过
- 判断是不是障碍
- 判断当前格是否允许朝该方向走出
- 判断目标格是否允许从该方向走入

只要这些条件都满足，就说明这一步合法：

- 更新 `dist[nx][ny] = dist[x][y] + 1`
- 把新格子入队

最后输出终点的 `dist` 即可。

容易错的地方

1. 只判断目标格能不能进，而忘了判断当前格能不能出。
2. 起点终点坐标是从 1 开始，数组是从 0 开始。
3. BFS 中“访问过”的判断必须在入队前做，否则会重复入队。

题目本质

这题是在上一题的基础上更进一步：

- 不要求最短距离
- 还要求输出整条最短路径

和 `bfs.cpp` 的区别

最短路长度只需要 `dist`。

但如果要恢复路径，就要记住：

- 每个点第一次被访问时，是从哪个点来的

所以新增了前驱数组 `pre`。

代码过程

前半部分和 `bfs.cpp` 一样，还是标准 BFS。

不同点在于，发现一个新点时除了更新距离，还做一件事：

- `pre[nx][ny] = cur`

这句话的意义是：

- `(nx, ny)` 这个点是从 `cur` 走过来的

当 BFS 结束后，如果终点不可达，输出 `-1` 和 `No path`。

如果终点可达，就从终点开始反向找：

- 当前点加入路径
- 跳到它的前驱
- 一直跳到起点之前的哨兵 `(-1, -1)`

这样得到的是“终点到起点”的逆序路径，所以最后要 `reverse` 一下。

为什么这样恢复一定是一条最短路

因为 BFS 第一次访问到一个点时，用的就是最短步数。

也就是说，`pre` 记录的是“这个点的最短路前驱”，所以最终回溯出来的是一条最短路径。

容易错的地方

1. `pre` 必须在第一次到达该点时记录，不能反复覆盖。
 2. 回溯路径时拿的是终点，不是起点。
 3. 反向回溯完成后一定要 `reverse`。
-

graph/color.cpp

题目本质

这道题是在问：

- 一个图最少需要多少种颜色，才能让相邻点颜色不同

这是一个典型的约束搜索问题。

为什么用 DFS + 回溯

因为每个点的颜色都要试，而一个点的选择会影响后面的点。

这种“逐个做决定、发现冲突就回退”的模式，就是标准回溯。

代码过程

第一步，读入图，用邻接矩阵 `adj` 存边关系。

第二步，枚举颜色数量 `k = 1...4`。

意思是：

- 先试试 1 种颜色能不能染
- 不行再试 2 种
- 再不行试 3 种
- 最后试 4 种

第三步，DFS 处理第 `current` 个点。

对当前点，尝试每一种颜色 `currentcolor`：

- 扫描所有邻居
- 如果有相邻点已经被染成同样颜色，说明冲突，不能选
- 如果没有冲突，就给当前点染上这个颜色，递归处理下一个点

如果后面失败了，就把当前点颜色恢复成未染色，继续试别的颜色。

当 `current == n` 时，说明所有点都成功染色，返回 `true`。

这题在练什么

它其实是在练一类通用模式：

- 方案构造
- 约束检查
- 冲突剪枝
- 回退恢复

容易错的地方

1. 染色失败后一定要恢复现场。
2. 未染色状态最好和合法颜色区分开，这里用的是 `-1`。
3. 邻接矩阵里如果是无向图，记得双向都设为 `true`。

graph/findAllisland.cpp

题目本质

这是最经典的“岛屿数量”问题，本质上就是统计二维网格中的连通分量个数。

为什么用 DFS

因为一旦找到一个陆地格子，就要把和它连成一片的整块陆地都找出来。

这正是 DFS/BFS 擅长的事。

代码过程

主函数 `numIsland` 会遍历整个网格。

如果看到一个值为 `'1'` 的格子，说明发现了一个新岛屿：

- 岛屿数加一
- 从这个格子开始 DFS

DFS 做的事情很简单：

- 把当前格子改成 '0'，表示已经访问过
- 再递归去找上下左右四个方向仍然是 '1' 的格子

这样一轮 DFS 会把整个连通块全部“吃掉”。

之后主循环继续扫网格时，这个岛的其他格子都已经变成 '0'，不会重复计数。

为什么能这样原地修改

因为这里不再需要保留原网格，所以直接把访问过的陆地改成水，既完成了标记，又省掉了 `visited` 数组。

容易错的地方

1. 网格为空时先判断。
2. DFS 前先把当前格子标记掉，否则会来回递归。
3. 题目如果改成八连通，就不能只看上下左右。

graph/maxpath.cpp

题目本质

这题是在矩阵中寻找最长严格递增路径，并且还要把路径本身保存下来。

为什么可以用 DFS + 记忆化

从某个格子只能走向更大的相邻格，所以路径值严格递增，不可能绕回到原点形成环。

这意味着搜索图本质上是一个 DAG 风格的结构，非常适合：

- DFS 搜索
- `dp[x][y]` 记忆化

代码过程

函数 `dfs(x, y)` 的含义是：

- 以 `(x, y)` 为起点，能得到的最长递增路径长度
- 以及这条最佳路径的具体内容

如果 `dp[x][y]` 已经算过，直接返回记忆结果。

否则就：

1. 先把当前格子的路径长度设为 1
2. 枚举四个方向
3. 如果下一个格子更大，就递归求它的最优答案
4. 从所有可行方向中选一个最优的子路径

最后把当前值放到子路径前面，组成从当前点出发的最优路径。

外层再枚举每个格子作为起点，取全局最优。

这题在练什么

它很适合帮助理解：

- DFS 不只是“能不能到达”
- 也可以“返回一个最优值”
- 如果有重复子问题，就配合记忆化

容易错的地方

1. 如果不做记忆化，会有大量重复搜索。
2. 这里比较的是“严格递增”，所以必须是 `>`，不能写成 `>=`。
3. 保存路径时，要注意当前点要插到子路径前面。

graph/shortestpathlen.cpp

题目本质

题目要的是：

- 访问所有节点
- 并且总路径长度最短

这已经不是普通“从 A 到 B 的最短路”了，因为“当前在哪个点”还不能完全描述状态，必须知道“哪些点已经访问过”。

为什么要引入 `mask`

如果只知道当前点，不知道已经访问了哪些点，就没法判断还差哪些点没访问。

所以完整状态必须写成：

- 当前所在节点 `u`
- 已访问节点集合 `mask`

例如 `mask` 的二进制第 `i` 位为 `1`，表示第 `i` 个点已经访问过。

为什么还是 BFS

虽然状态复杂了，但每走一步代价仍然相同，所以在“状态图”上仍然是无权最短路，还是可以用 BFS。

代码过程

第一步，把所有节点都作为起点同时入队。

这是因为题目没指定必须从哪个点出发，所以最优答案可能从任意点开始。

入队状态是：

- $(i, 1 \ll i, 0)$

表示当前在点 `i`，只访问过自己，路径长度为 `0`。

第二步，BFS 扩展状态：

- 从当前节点 `cur` 出发，枚举邻居 `next`
- 新状态的访问集合变成 `nextMask = mask | (1 << next)`

如果 $(next, nextMask)$ 没出现过，就入队。

第三步，什么时候结束？

当 `mask == (1 << n) - 1` 时，说明所有点都访问过了，这时当前步数就是最短答案。

这题在练什么

它本质在练：

- 扩展状态建模
- 位运算表示集合
- BFS 不一定跑在原图点上，也能跑在“状态点”上

容易错的地方

1. `visited` 必须按“点 + mask”一起判重。
 2. 所有点都要作为初始状态入队。
 3. `nextMask` 要用按位或，不是加法。
-

tree/re_serialize.cpp

题目本质

这题是在做两件互逆的事：

1. 把一棵树转成字符串
2. 再从字符串恢复出原树

为什么先序遍历 + `null` 可以唯一恢复

因为先序遍历顺序是：

- 根
- 左子树
- 右子树

如果空节点也显式写出来，那么整棵树的结构信息就完整保留下来了。

比如：

- 有值节点说明这里有根
- `null` 说明这里没有孩子

所以反序列化时只要按相同顺序把这串信息重新读一遍，就能把树恢复出来。

序列化过程

函数 `serialize` 做的是先序遍历：

- 如果当前为空，写入 `null`
- 否则先写当前节点值
- 再递归序列化左子树
- 再递归序列化右子树

反序列化过程

先把整个字符串按逗号切开，变成 token 数组。

然后用 `_deserialize(data, idx)` 递归建树。

这个函数的含义是：

- 从 `data[idx]` 开始，恢复一棵子树

步骤如下：

1. 取出当前 token，并去掉两端空格
2. 如果是 `null`，说明这里没有节点，`idx++` 后返回空指针
3. 否则创建一个新节点
4. 然后递归构造它的左子树
5. 再递归构造它的右子树

为什么 `idx` 必须按引用传递

因为整棵树恢复过程中，所有递归层都必须共享“当前读到了第几个 token”这一进度。

如果不共享，每层都会从自己的拷贝位置开始，整棵树就乱了。

容易错的地方

1. token 里可能带空格，`stoi` 前要先 trim。
2. `idx` 必须在处理当前 token 后及时递增。
3. 反序列化顺序必须和序列化顺序完全一致。

`tree/max_width_tree.cpp`

题目本质

题目要求的是二叉树最大宽度，而且宽度不是简单数这一层有几个实际节点，而是要把中间缺失的位置也算进去。

为什么普通层序遍历不够

如果只统计每层实际节点个数，那么遇到下面这种有空洞的树时会出错：

- 左边有一个深节点
- 右边也有一个深节点
- 中间缺了很多位置

题目中的宽度通常要求这些空位也算在跨度里。

为什么用位置编号

把树想象成嵌在一棵完全二叉树里：

- 根编号为 1
- 左孩子编号为 $2 * id$
- 右孩子编号为 $2 * id + 1$

这样同一层最左和最右节点的位置编号之差，就真实反映了跨度。

代码过程

队列里存的是：

- 节点指针
- 这个节点的位置编号

开始时根入队，编号为 1 。

每次处理一层之前：

- 当前层最左编号是 $q.front().second$
- 当前层最右编号是 $q.back().second$

所以当前层宽度就是：

- $right - left + 1$

然后把这一层的节点逐个弹出，并把它们的左右孩子按编号规则加入队列。

所有层处理完，取到的最大宽度就是答案。

容易错的地方

1. 宽度公式是 $right - left + 1$ ，不能漏掉 $+1$ 。
 2. 要在处理这一层之前就取 $front$ 和 $back$ ，因为进下一层后队列内容会变。
 3. 编号过大时在更大数据下可能需要用 `unsigned long long`，不过这份练习代码里数据量不一定大。
-

tree/longestdepth.cpp

题目本质

就是求一棵二叉树的高度。

为什么递归这么自然

因为一个节点的高度只依赖于：

- 左子树的高度
- 右子树的高度

所以它天然满足递归定义。

代码过程

对当前节点：

- 如果为空，深度是 0
- 否则递归求左深度和右深度
- 返回较大者加一

这其实就是典型的后序递归：

- 先求子树
- 再合并当前答案

这题的意义

它是很多树题的基础母题。直径、平衡树、路径问题都常常先从“求高度”这个递归模型变形而来。

tree/longestwayintree.cpp

题目本质

求树中最长路径，也就是通常说的直径。

最关键的思想

要把“递归返回值”和“最终答案”分开理解。

这里递归函数 `depth(root)` 返回的是：

- 从 `root` 出发，向下走，能得到的最长单边路径长度

但最终答案不一定是一条单边链，它可能：

- 从左子树某点上来
- 经过当前节点
- 再走到右子树某点

所以经过当前节点的最长路径长度是：

- $L + R + 1$

代码过程

对每个节点：

1. 递归求左子树深度 `L`
2. 递归求右子树深度 `R`
3. 用 $L + R + 1$ 更新全局答案
4. 返回 $\max(L, R) + 1$

为什么返回值不能直接写成 $L + R + 1$

因为父节点如果还想接上当前节点，只能从当前节点向下选一边继续走，不能左右两边都带上去。

所以返回给父节点的只能是一条单边最长链。

容易错的地方

1. 把“全局答案”和“递归返回值”混为一谈。
 2. 路径长度到底按边数还是按节点数计，这要看题目定义。这份代码按节点数在算。
-

tree/maxpathsum.cpp

题目本质

求一条路径，使路径上节点值之和最大。

为什么和直径题像

它和直径题的结构几乎一样：

- 也要看左边能给多少贡献
- 也要看右边能给多少贡献
- 也可能经过当前节点把左右拼起来

只是这次比较的不是长度，而是权值和。

代码过程

对每个节点：

1. 递归求左子树能提供的最大向下路径和
2. 递归求右子树能提供的最大向下路径和
3. 如果某边贡献是负数，就直接当成 0
4. 更新全局最大值为 `left + right + root->data`
5. 返回给父节点的是 `max(left, right) + root->data`

为什么负数贡献要舍弃

因为如果某条子路径和是负的，把它接上只会让总和更小，不如不要。

最核心的理解

- 更新全局答案时，可以左右两边都取
- 返回给父节点时，只能选一边

这和直径题是完全同一套路。

tree/buildfulltreewithfrontandback.cpp

题目本质

这题想做的是：

- 根据前序和后序遍历结果重构一棵树

为什么这题比“前序 + 中序”难

因为前序和后序通常不能唯一确定一棵普通二叉树。

要想唯一划分，往往要额外假设：

- 这是一棵满二叉树
- 或者每次划分都能被唯一确定

代码的思路

前序的第一个元素一定是当前子树的根。

然后：

- 前序里根后面的第一个元素，一定是左子树的根

知道了左子树根，就去后序里找它的位置。

找到后就能知道：

- 左子树一共有多少个节点

于是就能切分出：

- 前序里哪些属于左子树
- 哪些属于右子树
- 后序里哪些属于左子树

- 哪些属于右子树

再递归去构造。

这题为什么值得复习

虽然这份代码没写完，但它很适合帮助理解“树重构题的共性”：

- 先从某种遍历中确定根
 - 再借助另一种遍历划分左右子树范围
 - 再递归处理子问题
-

leetcode/寻找图中是否存在路径.cpp

题目本质

问无向图中 `source` 能不能到达 `destination`。

这其实不是最短路题，而是最基本的连通性判断。

为什么用 BFS

因为只需要知道“能不能到”，不关心具体路径长度，BFS 或 DFS 都行。

这份代码选择了 BFS。

代码过程

第一步，利用边表建邻接表。

第二步，准备 `visited` 数组和队列，把 `source` 入队。

第三步，循环弹出队头，扫描其所有邻居：

- 如果邻居就是终点，可以直接返回 `true`
- 如果没访问过，就标记并入队

最后如果 BFS 结束还没提前找到，就返回 `visited[destination]`。

为什么最后返回 `visited[destination]` 更稳

因为有一个特殊情况：

- `source == destination`

这时即使没有边，答案也应该是 `true`。如果只靠“遍历过程中看到邻居等于终点才返回 `true`”，就可能漏掉这种情况。

leetcode/不同颜色的路径.cpp

题目本质

要求从起点到各点的最短路径，但路径上的边颜色必须红蓝交替。

为什么普通最短路不够

因为“当前在某个点”还不够，你还必须知道：

- 你通过什么颜色的边到这里的

否则你没法判断下一步应该走红边还是蓝边。

所以状态怎么设计

状态写成：

- (节点, 上一条边颜色)

于是 `dist[u][0]` 和 `dist[u][1]` 表示两种不同状态下到达 `u` 的最短距离。

代码过程

先分别建立：

- 红边邻接表 `redGraph`
- 蓝边邻接表 `blueGraph`

初始化时把点 `0` 以两种颜色状态都入队，表示：

- 既可以认为“上一步是红”，下一步该走蓝
- 也可以认为“上一步是蓝”，下一步该走红

之后 BFS：

- 如果当前状态表示“上一步是红”，那下一步只能走蓝边
- 如果当前状态表示“上一步是蓝”，那下一步只能走红边

最后对每个点取两种状态中的较小值。

这题的本质训练点

是“最短路题里状态要不要扩一维”的判断能力。

leetcode/访问所有节点的最短路径.cpp

题目本质

和 `graph/shortestpathlen.cpp` 是同一题型。

它要求：

- 经过所有节点
- 并且总步数最小

为什么不能用普通单源最短路

因为有没有访问过某个点，会影响后续决策。

所以状态必须记录访问集合 `mask`。

代码过程理解

初始化把所有点同时入队，是因为：

- 可能从任意点起步都最优

每次出队一个状态 `(u, mask, length)`：

- 枚举 `u` 的邻居 `nu`
- 新状态集合变成 `mask | (1 << nu)`
- 如果这个状态没来过，就入队

一旦 `mask` 覆盖所有节点，当前长度就是答案。

复习时重点背什么

不是某一行代码，而是这一句思维转换：

- “原题的点” 变成 “状态图里的点”
-

leetcode/相邻字符的不同路径长度.cpp

题目本质

这是树上的最长路径问题，但加了一个限制：

- 相邻两个节点字符不能相同

为什么还是树形 DFS

因为图结构本身还是一棵树，只是合并路径时多了一个合法性条件。

代码过程

先根据 `parent[]` 建出无向树。

然后 DFS 每个节点 `u`，求：

- 从 `u` 向下出发，在满足字符不同限制下，最长能延伸多长

对每个子节点：

1. 先递归求出它能提供的最长链长度
2. 如果 `s[u] == s[child]`，这条链不能接到 `u` 上，直接跳过
3. 否则就用它更新当前节点能接出的前两大链长

最后：

- `ans = max(ans, firstLen + secondLen + 1)` 或当前实现的等价形式
- 返回 `firstLen + 1`

核心思想

它本质上仍是“树直径题”的变形，只不过在合并左右链之前要先检查约束是否合法。

leetcode/二叉树的直径.cpp

题目本质

和 `tree/longestwayintree.cpp` 完全同类。

应该怎么理解这题

对每个节点，真正有价值的是两个量：

1. 左子树向下最长链
2. 右子树向下最长链

于是：

- 经过当前节点的路径长度 = 左链 + 右链
- 返回给父节点的长度 = $\max(\text{左链}, \text{右链}) + 1$

这题和树高题的关系

树高题只要返回高度。

直径题是在“求高度”的基础上，顺手多维护一个全局最大值。

所以它是树高题的直接升级版。

leetcode/计算递增路径数量.cpp

题目本质

题目要求统计矩阵里所有严格递增路径的条数。

为什么可以按数值排序做 DP

如果一条路径是严格递增的，那么路径上后面的格子值一定更大。

所以当我们按值从小到大处理格子时：

- 当前格子的答案，只会依赖更小值格子的答案

这就很像 DAG 上的拓扑序 DP。

代码过程

第一步，把所有格子 (值, x , y) 收集起来。

第二步，按值从小到大排序。

第三步，初始化：

- $dp[x][y] = 1$

这代表：

- 只包含自己这个格子的路径，也算一条长度为 1 的递增路径

第四步，处理每个格子时，检查四邻域中所有值比自己小的格子：

- 如果邻居值更小，那么所有“以邻居结尾的递增路径”后面都可以接上当前格子
- 所以把 $dp[neighbor]$ 加到 $dp[x][y]$ 上

最后把所有格子的 dp 累加起来，就是答案。

这题在练什么

它在练：

- 图上的计数 DP
- 利用偏序关系进行转移
- “先排好可依赖顺序，再做动态规划”的思路